

Perfecting AI Agent Frameworks through Unified Design Principles

Sergio Lopez¹, Guillermo Ramos¹ and Asier Arranz²

¹*Kadmos Technologies GmbH, Germany*

²*NVIDIA, U.S.A.*

Keywords: AI Agent Frameworks, Software Architecture, Observability, Safety.

Abstract: Large language model (LLM) agents now power applications ranging from code generation to enterprise knowledge orchestration, yet available frameworks remain polarized. Research prototypes deliver transparent reasoning loops and pedagogical clarity, whereas production toolkits emphasize operational resilience, safety, and deployment tooling. This paper synthesizes these perspectives into a unified architecture that balances theory and practice. We articulate design principles centered on modularity, explainability, safety-by-default, and observability-first execution. Building on these principles, we propose a layered stack covering deliberation policies, orchestration engines, model abstraction, tool and service ecosystems, execution environments, and monitoring/governance planes. We illustrate how interoperable interfaces support cross-language implementations, how context propagation enables rich tool use and grounding, and how guardrails, evaluation pipelines, and deployment workflows can be embedded from the outset. The result is a pedagogical yet production-ready blueprint that educators, students, and developers can leverage to construct resilient agentic systems. We conclude with open research directions spanning evaluation benchmarks, self-improving policies, and multimodal orchestration, positioning the unified framework as a foundation for responsible innovation in the agent era.

1 INTRODUCTION

1.1 Motivation

Large language models have catalyzed a wave of agentic frameworks that promise autonomous task completion, multi-tool orchestration, and human-aligned reasoning loops (Russell and Norvig, 2020; Wang et al., 2022). Yet the pace of innovation has created a fractured landscape: research-oriented prototypes (e.g., SmolAgents (Hugging Face, 2024a)) emphasize first-principles reasoning and pedagogical clarity, open-source orchestration stacks decoupled from cloud platforms (e.g., LangGraph, AutoGen) provide bring-your-own-LLM flexibility, and production-grade toolkits maintained by cloud providers (e.g., Google’s Agent Development Kit) prioritize reliability, governance, and ecosystem integration (LangChain, 2024; Chen et al., 2023; Google LLC, 2025). Educators, students, and practitioners face a gap: no single reference coherently unites these perspectives into a reusable blueprint for designing robust AI agent frameworks.

1.2 Problem Statement

Despite overlapping goals, research prototypes and production toolkits embody divergent assumptions about runtime orchestration, safety scaffolding, and evaluation methodology. Prototype-oriented systems emphasize ReAct-based deliberation and memory formalism but often under-specify production considerations such as resumable executions or managed artifacts. Operational platforms excel at policy enforcement, streaming, and lifecycle management yet frequently defer theoretical grounding to ancillary guidance. Existing comparative studies of agent libraries (e.g., LangGraph, AutoGen, CrewAI) catalogue features rather than distill transferable design principles (LangChain, 2024; Chen et al., 2023; CrewAI Contributors, 2024). The field therefore lacks an educational yet actionable synthesis spanning cognition, architecture, and operational excellence.

1.3 Contributions

This paper addresses the gap by: (1) deriving a unified architectural stack that harmonizes theoretical scaffolding with production runtimes; (2) presenting a

methodology for aligning deliberative control loops, tool ecosystems, and context propagation across heterogeneous implementations; (3) codifying safety, evaluation, and deployment patterns that transform research prototypes into production-ready systems; and (4) providing pedagogical artifacts—notation tables, algorithm summaries, and comparative analyses—that instructors and practitioners can adapt for course-work or technical migrations.

1.4 Paper Organization

Section 2 reviews foundational cognitive models, summarizes reference frameworks, and positions related work. Section 3 introduces the unified architecture, while Section 4 expands on runtime semantics and service layers. Sections 5 and 6 examine tooling, grounding, safety, and governance. Section 7 details observability and evaluation workflows, and Section 8 outlines deployment and lifecycle considerations. Section 9 discusses lessons learned and open research directions, and Section 10 concludes.

2 BACKGROUND AND RELATED WORK

2.1 Cognitive Foundations of Agentic Systems

Modern agent frameworks build upon decades of research in intelligent agents, bounded rationality, and perception–action cycles (Russell and Subramanian, 1995). Russell and Norvig formalized the agent–environment interface, offering a taxonomy that remains foundational for contemporary designs (Russell and Norvig, 2020). Brooks’ subsumption architecture and subsequent hybrid control models demonstrated the value of layered reasoning and reactive feedback (Brooks, 1986; Gat, 1998). ReAct (Reason + Act) blends deliberative thought with tool-grounded action, yielding transparent transcript history¹ and policies $\pi(a_t | h_t)$ that inspire today’s LLM-centric agents (Wang et al., 2022; Hugging Face, 2024b). Complementary theories on working memory (Baddeley and Hitch, 1974), symbol grounding (Harnad, 1990), free-energy minimization (Friston, 2010), cognitive architectures such as ACT-R (Anderson, 1996), and unified cognition proposals (Gibson, 1979; Newell, 1990; Simon, 1955; Albus, 1991)

¹The term “transcript history” refers to the structured record of all reasoning and action steps an agent emits throughout an invocation.

further motivate explicit memory modules, environmental grounding, and self-regulation loops in agent frameworks.

2.2 Research-Oriented Agent Prototypes

Open-source research prototypes focus on teaching foundational principles by pairing formal notation with accessible case studies (Wang et al., 2022; Hugging Face, 2024b; Hugging Face, 2024a; LangChain, 2024; Chen et al., 2023). Common traits include explicit modeling of the deliberation loop, richly annotated memory transcripts, and mathematical framing of control policies. These systems often provide migration guides or language-portability notes to encourage reimplementations, yet typically defer infrastructure concerns such as artifact storage, long-running workflows, or managed deployment targets.

2.3 Production-Grade Agent Toolkits

Industry-maintained toolkits emphasize operational robustness, focusing on event-driven runtimes, standardized service abstractions, and integrations with existing cloud ecosystems (Google LLC, 2025). They commonly introduce context objects that package session history, identity, and policy metadata; workflow controllers that support sequential, parallel, or looped execution without additional LLM inference; and plugin architectures for cross-cutting concerns such as logging, caching, and guardrails. While feature-rich, these platforms frequently assume the reader already understands cognitive underpinnings, leaving an implicit gap between conceptual models and practical implementation.

2.4 Comparative Landscape

Several open-source ecosystems attempt to bridge research and practice. LangGraph models agent workflows as cyclic graphs with tool nodes and conditional routing, emphasizing deterministic control but requiring manual safety instrumentation (LangChain, 2024). AutoGen automates multi-agent role assignments and conversation-driven planning, yet its evaluation and deployment guidance remain nascent (Chen et al., 2023). CrewAI focuses on role-based collaboration and human hand-off patterns but offers minimal runtime formalization (CrewAI Contributors, 2024). These initiatives underscore the appetite for modular, multi-agent platforms while highlighting the absence of a rigorous, pedagogical framework that integrates cognitive theory, runtime services, and opera-

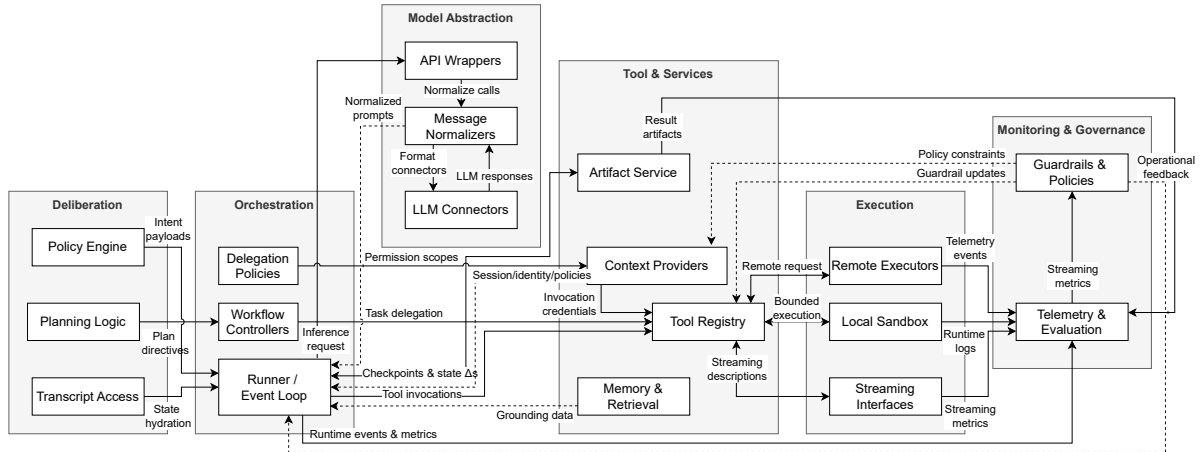


Figure 1: Unified architecture showing the contracts exchanged across deliberation, orchestration, model abstraction, tooling, execution, and governance layers.

tional governance. The unified approach proposed in this paper aims to fill that gap.

Appendix D (Table 8) summarizes these frameworks side-by-side for readers interested in a detailed comparison of abstractions, strengths, and remaining gaps.

3 UNIFIED AGENT FRAMEWORK ARCHITECTURE

3.1 Architectural Principles

The proposed blueprint embraces four design tenets. **Modularity** demands that agent cognition, tooling, execution, and governance layers can evolve independently while exposing stable interfaces. **Explainability** elevates transparent transcripts, introspectable state, and reproducible trajectories rooted in the deliberative reasoning literature (Russell and Norvig, 2020; Wang et al., 2022). **Safety-by-default** requires that every layer accepts policy hooks and identity controls so that new capabilities inherit guardrails rather than bolt them on. Finally, an **observability-first** posture assumes each component emits structured telemetry, enabling continuous evaluation and debugging (Google LLC, 2025). These principles harmonize academically grounded agent modeling with the operational experience of modern toolkits, providing a scaffold that can accommodate future innovations without architectural upheaval.

3.2 Layered Model Overview

Figure 1 depicts the consolidated stack: (1) a **Deliberation Layer** that encapsulates policy prompts, planning algorithms, and reasoning traces; (2) an **Orchestration Layer** that coordinates agent hierarchies, workflow controllers, and delegation policies; (3) a **Model Abstraction Layer** wrapping heterogeneous LLM endpoints with normalized messaging; (4) a **Tools & Services Layer** unifying tool registry, context propagation, artifact management, and memory retrieval; (5) an **Execution Layer** spanning local and remote sandboxes, streaming channels, and resumable processes; and (6) a **Monitoring & Governance Layer** aggregating telemetry, guardrails, and evaluation feedback loops. Each layer publishes contracts to the layers above and below, ensuring that implementations can swap components without perturbing the overall system. Table 1 enumerates representative interface mappings for Python and Go ecosystems.

3.3 Component Interoperability

Interoperability depends on a shared vocabulary of interfaces. Algorithm 1 (Appendix B) details how orchestration components delegate work across these interfaces while respecting permissions and policies. The deliberation layer exposes a ‘PolicyEngine’ contract that accepts transcript histories h_t and returns structured intents, enabling orchestration engines to remain agnostic to the underlying LLM or planner (Wang et al., 2022). Orchestration components publish an ‘InvocationDescriptor’ describing agent routing, tool permissions, and escalation rules. Tools conform to a standard schema—name, description, JSON schema for inputs/outputs, optional streaming callbacks—so that registries and validators can

Table 1: Cross-language Interface Mappings.

Interface Concept	Python Implementation	Go Implementation	Notes
PolicyEngine	Abstract class with <code>generate</code> , <code>plan</code> coroutines	Interface with <code>Generate</code> , <code>Plan</code> methods returning structs	Supports async vs. goroutine semantics
InvocationDescriptor	Dataclass describing routing, permissions	Struct with routing fields validated via tags	Serialized via JSON/Proto
ToolRegistry	Map of callable adapters with Pydantic schemas	Map of tool structs implementing <code>Execute(context)</code>	Shared schema definitions (MCP-compliant tool manifest)
ContextProvider	Dependency-injected service accessor	Interface returning service handles	Aligns lifecycle hooks (Before/After)
Event	Dataclass persisted via JSONL	Struct with <code>MarshalJSON</code> for storage	Common schema ensures replay parity
ArtifactService	Async class with <code>put/get</code> enforcing namespaces	Interface backed by object storage client	Retention policies handled externally

reason about capabilities uniformly (Google LLC, 2025). Services implement ‘ContextProviders’ that attach session metadata, identity credentials, memory handles, and artifact references to each invocation. By treating transcripts, events, tools, and services as typed artifacts, mixed-language implementations (Python, Go, Java) can interoperate via serialization formats such as JSON or Protocol Buffers without sacrificing expressiveness.

3.4 Agent-to-Agent Interoperability

Beyond component-level integration, emerging ecosystems increasingly demand communication across autonomous entities. Agent-to-agent (A2A) interoperability extends the unified architecture’s interface vocabulary to enable structured dialogue, coordination, and delegation among agents developed under heterogeneous frameworks. Google’s Agent-to-Agent patterns, as documented by Gulli (Gulli, 2025), formalize message schemas, intent negotiation, and trust boundaries so that agents can coordinate directly across organizational or runtime limits while preserving auditability. In parallel, shared tool vocabularies such as the Model Context Protocol (MCP) (Anthropic, 2025b; Zhang et al., 2025) let agents invoke external systems and resources through standardized tool contracts, complementing but not replacing A2A messaging. Within the unified blueprint, A2A channels manifest as higher-order extensions of the orchestration layer, bridging isolated runtimes into federated networks capable of negotiation, specialization, and mutual supervision. Aligning these exchanges with the same telemetry, policy enforcement, and sandboxing hooks applied to internal components ensures that governance remains enforceable even as agents collaborate in distributed ecosystems.

3.5 Reference Implementations

Table 1 summarizes how these interfaces materialize across representative technology stacks. In a Python environment, the orchestration layer may map to an event-driven runner paired with coroutine-based agents, while tools and services leverage asynchronous context managers and dependency injection. In a Go implementation focused on systems-level determinism, the same interfaces translate into goroutine schedulers, channel-based event emission, and strongly typed tool registries. The abstraction boundaries ensure both ecosystems can share evaluation datasets, safety plug-ins, and monitoring pipelines, fostering portability for research prototypes and production deployments alike.

4 RUNTIME, STATE MANAGEMENT, AND SERVICES

4.1 Event-Driven Execution Model

The runtime stitches deliberation and action into a deterministic event loop. Each invocation begins with a runner seeding an initial transcript h_0 comprising system prompts, user goals, and environmental metadata (Wang et al., 2022). The policy engine generates structured intents—thoughts, tool calls, control directives—which the runner serializes into events. Every event is processed exactly once because the runner persists the log, checkpoints state before acknowledging tool outputs, and replays only the most recent committed event identifiers while enforcing idempotent side effects (Google LLC, 2025; Beyer et al., 2016). This mirrors the ReAct pseudocode yet extends it with resumability: should a long-running

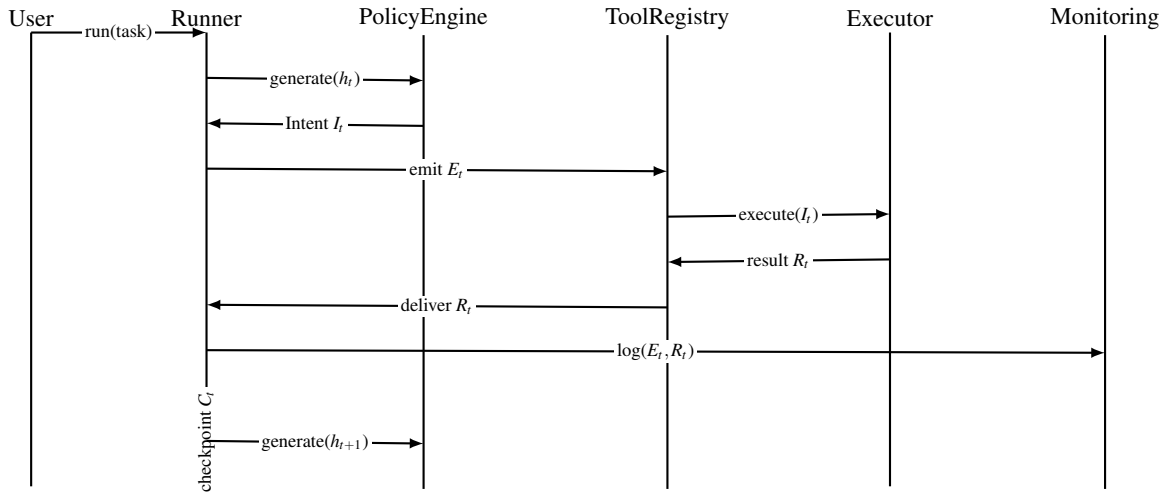


Figure 2: Event-driven runtime sequence illustrating intent generation, tool execution, telemetry logging, and checkpoint persistence.

tool suspend execution, the runner persists the event log so it can resume from the last committed checkpoint (Google LLC, 2025). To shield upstream services from bursty workloads, the runtime also enforces backoff-capable rate limiters that apply exponential delays with decorrelated jitter so noisy tool APIs remain stable under load (Brooker, 2015). Figure 2 contrasts the canonical ReAct loop with the enriched event-driven runtime, highlighting complexity contributions from language model latency, tool execution cost, and service round trips. Algorithm 2 (Appendix B) formalizes this loop, including checkpoints and resumable execution semantics.

4.2 Context Propagation

To keep agents stateless and composable, the runtime injects explicit context objects into every callback. An ‘InvocationContext’ bundles the invocation identifier, session handles, credential brokers, tracing metadata, and configuration flags. ‘CallbackContext’ narrows that view for before/after hooks, allowing policies or plugins to mutate prompts, enforce quotas, or short-circuit tool calls (Google LLC, 2025). Tool handlers receive a ‘ToolContext’ that grants access to session state, artifact stores, and authentication helpers without leaking internal runner details. This disciplined propagation ensures that every component can access the data it needs—yet only within its security scope—supporting auditable pipelines and clean dependency boundaries.

4.3 Session, State, and Memory Management

Session management unifies short-term working memory with long-term knowledge bases (Baddeley and Hitch, 1974; Harnad, 1990; Knill and Pouget, 2004). Implementations such as the ‘AgentMemory’ ledger append structured ‘MemoryStep’ entries for every planning or action turn, keeping transcripts replayable and auditable. Each session stores an ordered list of events plus a mutable state map representing the agent’s working set (e.g., intermediate calculations). Updates occur via event actions, guaranteeing replayability and traceability. Long-term memory connects to vector stores, knowledge graphs, or document indexes, but interactions are mediated through query APIs that record provenance and enforce data minimization policies. Table 2 enumerates the core context artifacts—transcripts, state deltas, and memory queries—alongside lifecycle hooks (`before_agent`, `after_tool`, evaluation checkpoints) to aid implementers in mapping responsibilities.

4.4 Artifact and Resume Services

Rich agent workflows frequently manipulate binary payloads (reports, images, code bundles). The artifact service version-controls these assets, associates them with sessions or users, and exposes streaming interfaces for incremental uploads. The artifact-service layer applies namespaces to prevent cross-

²Memory queries conform to the same context schema as the Tool Context, exposing provenance fields and access controls through standard invocation metadata.

Table 2: Context Artifacts and Lifecycle Hooks in Session Management.

Context	Field	Purpose
Invocation Context	session_service	Persist transcripts and mutable state
	artifact_service	Manage files and binaries within namespaces
	memory_service	Access long-term knowledge stores
	identity	Track acting principal (agent or user)
	policies	Apply guardrails and constraint sets
Callback Context	agent_name	Identify agent emitting callback
	hook_type	Distinguish before/after agent, model, or tool hooks
	override	Allow short-circuiting or modifying execution
Tool Context ²	invocation_context	Link back to invocation metadata
	credentials	Provide scoped tokens for external APIs
	session_state	Surface mutable per-invocation state snapshot

session leakage, ensuring each stored object inherits the session or user identifier from its invocation context. This guarantees that tools and resumable workflows can only access artifacts within their authorized scope. Retention policies and access logs further support compliance reviews and provenance audits. Complementary resume services capture the full invocation snapshot—event log, pending tool requests, locked artifacts—enabling human supervisors to intervene or continue execution after a pause. Together, artifact and resume services transform the agent runtime from a stateless chat loop into a stateful workflow engine suitable for regulated domains (Google LLC, 2025).

5 TOOLING, GROUNDING, AND EXTERNAL INTEGRATIONS

5.1 Tool Taxonomy

A unified tool registry categorizes capabilities by invocation semantics. *Synchronous tools* execute within a single event, ideal for lightweight APIs and deterministic computations. *Long-running tools* emit intermediate checkpoints, allowing progress tracking and cancellation hooks, while *streaming tools* push in-

cremental data (audio frames, market feeds) that the agent can react to in real time (Google LLC, 2025). Each tool declaration includes human-readable descriptions, JSON schemas for inputs and outputs, optional authentication scopes, and safety metadata (rate limits, confirmation requirements). Validation occurs at registration time to catch schema inconsistencies, and before execution to enforce policy constraints. At runtime, ‘ToolCollection’ brokers these contracts so agent policies can enumerate, filter, and call tools dynamically. Table 3 summarizes latency expectations, guardrail considerations, and representative examples for each tool category. Specialized code-generation agents illustrate these patterns, pairing large context windows with repository-aware guidance to ensure tool calls comply with project conventions (OpenAI, 2021; Wikipedia, 2025; OpenAI Academy, 2024).

5.2 Agent-as-Tool and Delegation Patterns

Complex applications orchestrate multiple agents as reusable capabilities. Delegation patterns range from hierarchical planners that spawn specialist agents to workflow controllers executing deterministic pipelines without additional LLM calls (LangChain, 2024; Anthropic, 2024). To keep wiring tractable, the unified framework treats agents as first-class tools: a `DelegationDescriptor` records target agent identity, expected contract, and permissible mutations to shared state. Figure 3 visualizes common topologies—tree-structured managers, graph-based collaborations, and Model Context Protocol (MCP) bridges that expose external ecosystems through standardized channels—highlighting how orchestration policies maintain accountability and traceability even as responsibilities propagate across agents (Anthropic, 2025b; Zhang et al., 2025). These agent-as-tool bridges operate hand in hand with the agent-to-agent interoperability channels outlined in Section 3, ensuring that delegation within a framework can escalate into protocol-governed collaboration across frameworks when required. As emphasized by Gulli (Gulli, 2025), the agent-as-tool paradigm naturally extends into the broader Agent-to-Agent (A2A) communication pattern, where localized delegation evolves into network-level cooperation. In this view, a `DelegationDescriptor` is conceptually analogous to an *Agent Card*³ in A2A, encapsulating identity, capabilities, and communication endpoints. By aligning the unified framework’s delegation model with A2A open standards and the Model Context Proto-

³An Agent Card is the standardized identity and capability descriptor that lets one agent discover, authenticate, and interact with another agent through a shared protocol.

Table 3: Tool Categories and Safety Considerations.

Category	Invocation Pattern	Latency Profile	Guardrail Considerations	Examples
Synchronous	Single event, immediate response	Low	Input validation, rate limiting	Search API, calculator
Long-running	Intermediate updates, deferred completion	Medium / variable	Cancellation support, progress auditing	Data pipeline run, report generation
Streaming	Continuous bidirectional data	Variable	Bandwidth quotas, interruption handling	Audio transcription, live monitoring
Delegated Agent	Scoped sub-agent invocation	Variable	Permission scoping, audit trails	Specialist planner, code reviewer
Code Execution	Sandbox execution of generated code	Medium	Sandbox policies, output sanitization	Python interpreter, SQL executor

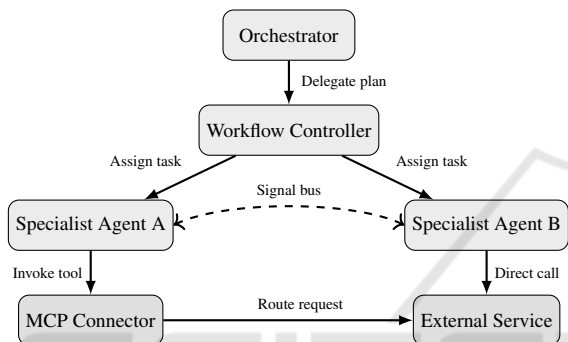


Figure 3: Delegation topology with orchestrator-managed planning, MCP-mediated tool access, and collaboration between specialist agents.

col (MCP) (Anthropic, 2025b), agents can invoke one another as callable tools. They can also participate in cross-framework ecosystems that support discovery, streaming exchanges, and multi-turn task coordination. This convergence reflects the *inter-agent composition* and *collaborative exploration* patterns, positioning the unified architecture as a practical substrate for distributed, protocol-compliant agentic systems.

5.3 Grounding Data and Retrieval Systems

Grounded reasoning demands access to verified knowledge bases. The framework separates session-level state (recent context), retrieved knowledge (documents, database rows), and artifact-derived data (files produced mid-run). Retrieval flows pass through adapters that handle authentication, query planning, and result normalization before re-injecting findings into the transcript with provenance tags (Harnad, 1990; Friston, 2010; Anthropic, 2025a). This discipline guards against prompt injection, enables selective disclosure policies, and supports caching strategies tuned to specific data domains (e.g., structured SQL vs. unstructured documents). Developers can

plug in vector stores, database connectors, or third-party MCP servers without rewriting agent logic.

5.4 Code Execution and Sandbox Strategies

Code execution extends agent capabilities but amplifies risk. The execution layer therefore offers tiered sandboxes: in-process interpreters with AST filters for lightweight tasks, containerized or serverless backends for heavier workloads, and isolated cloud executors for regulated environments (Google LLC, 2025; OpenAI, 2021; Wikipedia, 2025; OpenAI Academy, 2024). In practice, ‘PythonExecutor’ adapters dispatch code to local or remote runtimes while reporting logs and artifacts back into the transcript. Policies govern import whitelists, resource quotas, and outbound network access, while audit logs capture source code snippets, runtime metadata, and outputs for review, drawing on decades of compiler and programming-language safety practice (McCarthy, 1960; Knuth, 1965; Naur, 1963; Aho et al., 2006). By exposing execution as another tool type—with explicit contracts and guardrails—the framework preserves composability without sacrificing security.

6 SAFETY, SECURITY, AND GOVERNANCE

6.1 Threat Model and Risk Assessment

Robust agent frameworks begin with an explicit threat model. Key risk categories include goal misalignment (agents pursuing proxy objectives), harmful content (toxicity, bias, disallowed topics), unsafe actions (unauthorized system changes, financial transactions), data exfiltration (unintended disclosure of sensitive information), and prompt injection originating from

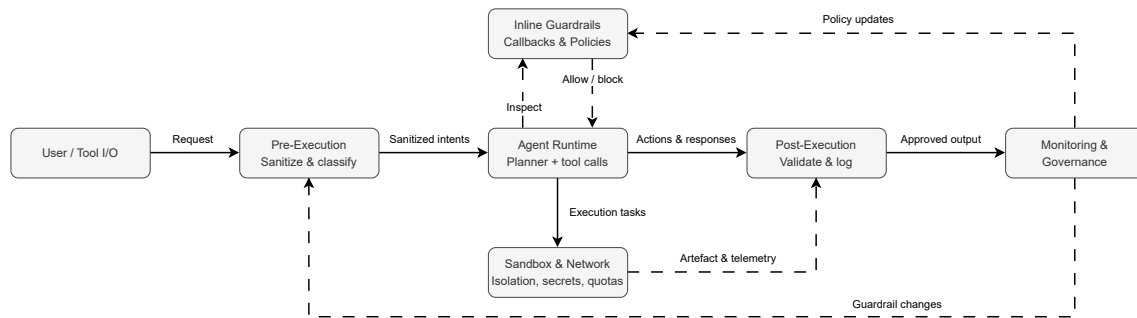


Figure 4: Defense-in-depth guardrail flow: requests enter pre-execution checks, encounter inline policies around the runtime, execute under sandboxed controls, and finalize with post-execution validation and governance feedback.

end users or retrieved documents (Google LLC, 2025; ClaudeAI.wiki, 2025; Eureka by Patsnap, 2024). Engineers weigh these risks against contextual factors such as deployment domain, user population, connected systems, and regulatory constraints before enabling autonomous behaviors. Structured hazard analyses and tabletop simulations help enumerate failure modes ahead of any autonomy increases.

6.2 Identity and Authorization Controls

Identity governs what an agent can do. Two complementary models dominate: service identity (agent-auth) uses a managed service account for all tool calls, simplifying access control but conflating user attribution; delegated identity (user-auth) leverages tokens or signed requests provided by the initiating user, preserving least privilege at the cost of additional consent flows (Google LLC, 2025; Anthropic, 2025a). The unified framework supports both by integrating credential brokers within context providers. Every tool execution records the acting identity, requested scopes, and authorization verdict, making post-event auditing straightforward. Secrets are vaulted, rotated automatically, and injected just-in-time into sandboxed environments to minimize exposure.

6.3 Guardrail Architecture

Figure 4 illustrates a layered defense-in-depth strategy. Pre-execution guardrails sanitize user inputs, enforce policy templates, and leverage lightweight classifiers or secondary LLMs for content filtering (Wang et al., 2022; Google LLC, 2025; ClaudeAI.wiki, 2025). Inline guardrails harness callbacks and plugins to inspect model requests, tool arguments, and responses in real time, aborting or modifying actions if they violate constraints—an approach aligned with metacognitive control architectures and interpretability science (Cox and Ram, 1995; Doshi-Velez and Kim, 2017). Post-execution guardrails validate results (e.g., checking database mutations against

whitelists), update incident counters, and trigger notifications when thresholds are exceeded. Sandboxing limits code execution privileges, while network micro-segmentation and VPC boundaries constrain lateral movement. Policies remain declarative and versioned so engineering and compliance stakeholders can co-manage them, and constitutional alignment pipelines demonstrate how those policies can be compiled into self-critique and revision loops before deployment (Bai et al., 2022).

6.4 Human-in-the-Loop and Compliance

Even with automated safeguards, certain scenarios warrant human oversight. Confirmation workflows require explicit approval before high-impact tool invocations (financial transfers, infrastructure changes). Escalation paths allow agents to yield control, handing the transcript, state snapshot, and pending actions to a human operator who may approve, modify, or cancel the plan. Compliance features—PII masking, data retention schedules, geographic residency controls—are enforced at the service layer, ensuring consistent application across agents and tools (Anthropic, 2025a). Audit trails combine event logs, guardrail decisions, and confirmation records, supporting internal reviews and external regulatory audits.

7 OBSERVABILITY, EVALUATION, AND QUALITY ASSURANCE

7.1 Telemetry and Traceability

Observability begins with rich event logging. Every event emitted by the runtime carries a correlation identifier (invocation ID), causal links (parent event IDs), author metadata, and action payloads. These logs feed

Table 4: Metrics Catalog with Collection Points.

Metric	Definition	Collection Point	Alert Threshold Guidance
Interactive latency	Time from user request to final answer for interactive flows	Runner completion	$P_{95} > 10\text{ s}$ or $P_{99} > 20\text{ s}$
Workflow latency	Completion time for long-running workflows compared to SLA baseline	Orchestration / execution layer	P_{95} exceeding SLA by $\sim 25\%$
Step latency (model)	Time spent in LLM inference per step	Policy engine wrapper	P_{99} exceeding baseline by 30%
Tool success rate	Successful tool executions / total calls	Tool registry	Ratio $< 95\%$ signals degradation
Guardrail interventions	Count of blocked/modified actions	Callbacks, plugins	Sudden spikes flag policy issues
Token usage	Tokens consumed per invocation	LLM client	Alerts on quota thresholds <i>per service</i>
Evaluation score	Aggregate nightly evaluation metric	Evaluation pipeline	Failing scores block deployment

both real-time dashboards and long-term data lakes where analysts can reconstruct trajectories. Figure 5 maps the flow from runtime emitters through message buses to monitoring backends such as logging systems, tracing platforms, or specialized agent observability tools. Structured logging formats (JSON, Protobuf) allow downstream processors to compute derived metrics, attach labels (user segment, agent version), and flag anomalies without parsing brittle text blobs (Google LLC, 2025; Beyer et al., 2016).

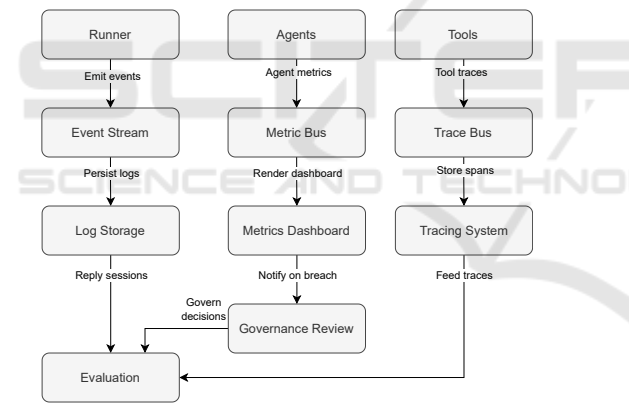


Figure 5: Observability data flow from runtime components to logging, monitoring, tracing, and evaluation pipelines.

7.2 Metrics and Monitoring

Operational visibility depends on a concise yet expressive metric set. Core performance metrics include end-to-end latency, step latency (model vs. service), token consumption, and tool success rate. Reliability metrics track error rates, retries, and rollback counts, while safety metrics monitor guardrail interventions, confirmation deferrals, and policy violations. Engagement metrics capture user satisfaction signals gathered through implicit (session length, repeated confirmations) or explicit feedback. Plugins and callbacks emit these metrics at decision points so monitoring systems can alert on threshold breaches,

drawing on decades of production telemetry and networking practice (Metcalf and Boggs, 1976; Jacobson, 1988; Brooker, 2015; Turner, 1986; Lam and Shankar, 1984). Table 4 aligns each metric with its collection locus (runner, tool adapter, guardrail) and recommended alert thresholds.

7.3 Evaluation Methodology

Automated evaluation augments production monitoring by introducing offline quality gates that quantify behavioral consistency and policy adherence. Two complementary data modalities are recommended: (1) unit-style test cases that replay short interactions to verify tool invocation sequences or deterministic responses, and (2) evaluation sets composed of extended dialogues with annotated trajectories and reference outcomes suitable for regression analysis or managed evaluation services (Google LLC, 2025). Beyond exact matching, evaluators measure precision and recall over action lists, compare plan structures, and use LLM-graded judgments to assess open-ended or creative outputs. Evaluation pipelines operate on regular schedules (e.g., nightly) and on demand before major releases, producing quantitative scorecards that inform governance reviews and drive iterative refinement. Algorithm 3 (Appendix B) formalizes the evaluation workflow described in this section.

7.4 Debugging and Replay

Deterministic replay transforms logs into actionable insights. Developers can load a session transcript, step through each event, inspect context objects, and re-execute tool calls in a sandbox to reproduce issues. The framework surfaces diff views comparing current behavior with prior baselines, highlighting divergences in reasoning tokens or tool parameters. Developer tooling—command-line utilities, notebooks, or web UIs—layers these capabilities atop the event log without requiring custom instrumentation. When

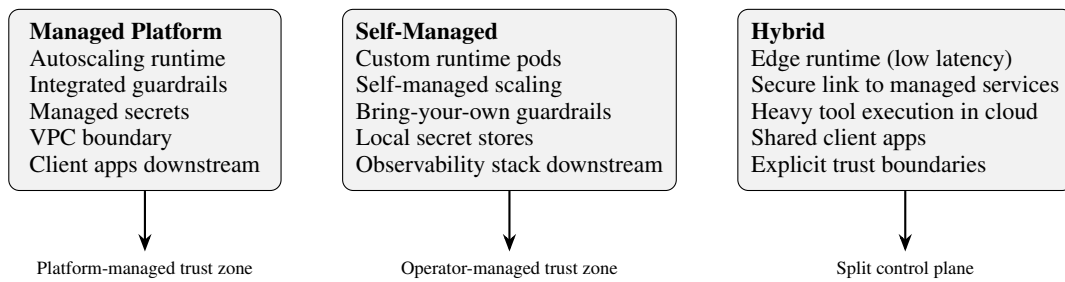


Figure 6: Managed, self-managed, and hybrid deployment patterns with indicative trust boundaries and service placement.

coupled with artifact snapshots and guardrail traces, replay allows for rapid diagnosis of hallucinations, latency regressions, or policy misconfigurations.

8 DEPLOYMENT, SCALABILITY, AND LIFECYCLE MANAGEMENT

8.1 Deployment Targets

Deployment choices balance velocity, compliance, and operational control. Managed platforms provide auto-scaling runtimes, secret management, and integrated observability, enabling teams to focus on agent logic while inheriting platform guardrails (Google LLC, 2025). Self-managed options—container services, Kubernetes clusters, or on-premise bare metal—offer fine-grained control over networking, hardware accelerators, and data residency at the expense of greater operational overhead. Hybrid topologies often emerge: latency-sensitive components run close to users, while heavy tool execution or retrieval services reside in secure cloud enclaves. Figure 6 compares common layouts, highlighting trust zones and data perimeters relevant to regulated industries.

8.2 CI/CD and Configuration Management

Continuous delivery pipelines treat agent configurations (prompts, tool registries, policies) as versioned artifacts alongside code. Infrastructure-as-code templates provision runtime environments, managed services, and networking policies in a reproducible manner. Secrets and credentials live in centralized vaults, referenced via environment-specific bindings injected at deploy time. Promotion flows progress from local sandboxes to staging environments with realistic data mocks, culminating in production rollouts guarded by canary evaluations and guardrail monitoring. Auto-

mated checks ensure evaluation suites pass and safety policies remain intact before release.

8.3 Scalability Considerations

Scaling agent systems requires holistic resource planning across compute, tooling, and state management layers. Compute scaling involves provisioning LLM endpoints with adaptive concurrency and batching, possibly across multiple model providers to balance cost and latency (Dean et al., 2013; Liang et al., 2024). Tool scaling ensures that external services (databases, search indexes, execution sandboxes) can handle bursty workloads without throttling agents; load-shedding strategies and circuit-breaker patterns prioritize critical requests when capacity tightens (Christensen and Maurer, 2014). State scaling addresses session growth by sharding transcripts, pruning stale sessions, and archiving artifacts according to retention and compliance policies (Zaharia et al., 2010; DeCandia et al., 2007). Backpressure signals propagate through the event loop so agents can degrade gracefully—switching to summary responses or deferring non-critical tasks when resources are constrained (Carbone et al., 2015). Together, these mechanisms preserve responsiveness and reliability under dynamic workloads while remaining consistent with safety-by-default and observability-first design principles.

8.4 Operations and Maintenance

Long-lived deployments demand proactive operations. Runbooks codify responses to incidents such as model outages, tool failures, or guardrail escalations. Health checks validate dependencies (LLM availability, memory connectors, artifact stores) and feed into auto-remediation workflows. Scheduled maintenance windows support policy updates, model upgrades, and security patches, each accompanied by regression evaluations and staged rollouts. Post-incident reviews incorporate event logs, guardrail decisions, and user impact analyses to refine playbooks and improve resilience over time.

9 DISCUSSION AND FUTURE DIRECTIONS

9.1 Comparative Insights

The unified framework demonstrates that research prototypes and enterprise toolkits converge on common abstractions when viewed through the lens of deliberation loops, event-driven orchestration, and service layers. Research systems contribute rigorous reasoning formalisms, transparent memory constructs, and pedagogical clarity; production platforms add hardened runtimes, governance hooks, and deployment-ready tooling. Integrating these perspectives reveals trade-offs: richer theoretical formalisms can be retained without compromising operational guarantees, while production guardrails must be expressible in the same formal language that governs policy and planning. The synthesis also underscores the importance of context propagation—both token-level (long transcripts) and meta-level (identity, policy, provenance)—as the connective tissue across layers (Russell and Norvig, 2020; Google LLC, 2025; Anthropic, 2025a). Extending interoperability beyond intra-framework components to agent-to-agent exchanges via emerging A2A protocols (e.g., Google’s Agent-to-Agent specification) shows how unified contracts can span organizational boundaries while retaining auditability and policy control (Gulli, 2025).

9.2 Emerging Trends

Four directional trends stand out. First, evaluation is becoming agent-specific, with benchmarks that capture trajectories, tool decisions, and compliance behaviors rather than static outputs; continuous evaluation infrastructure must therefore ingest event logs and guardrail telemetry. Second, self-improving policies—whether via program synthesis, rule induction, or reinforcement on simulated environments—will pressure frameworks to support safe experimentation, policy versioning, and rollback mechanisms, while emerging constitution-based alignment methods couple these updates with governance corpora that keep autonomy bounded (Bai et al., 2022). Third, agent-to-agent protocols are maturing into open standards that enable negotiation, delegated execution, and shared tool marketplaces across heterogeneous runtimes. Finally, multimodal and embodied orchestration is moving from novelty to necessity as agents interpret audio, video, and sensor data; the unified architecture needs to accommodate asynchronous modalities, streaming inference, and cross-modal grounding without breaking existing contracts.

9.3 Open Research Questions

Despite progress, several questions remain open. What formal guarantees can be offered for agents operating with non-deterministic LLM outputs when combined with deterministic workflow controllers? How can constitutional or policy-based alignment methods be translated into verifiable runtime checks that satisfy regulatory requirements? What models best capture collaborative dynamics between human supervisors and autonomous agents, particularly when responsibility is shared across delegation trees? How can long-context reasoning be balanced with privacy and data minimization obligations when transcripts span sensitive domains? Finally, what verification and trust models can ensure that federated agent-to-agent ecosystems—mediated by constructs such as Agent Cards and shared registries—remain authentic, auditable, and governable across organizational boundaries?

9.4 Limitations

This study derives a generic architecture from publicly documented practices and observed design patterns across existing frameworks, but it does not provide empirical benchmarks or case studies validating every component. Certain innovations—such as proprietary safety classifiers, closed-source deployment platforms, or domain-specific compliance frameworks—remain under-described in public literature and may require bespoke integration. Additionally, the framework assumes access to reliable logging, identity management, and sandbox infrastructure; practitioners operating in resource-constrained or offline environments will need to adapt portions of the blueprint accordingly. Finally, emerging modalities (e.g., robotics control, tactile feedback) may demand extensions beyond the scope of this paper’s abstracted layers.

10 CONCLUSION

This work presents a unified architecture for AI agent frameworks that reconciles theoretical rigor with production-grade execution. By layering deliberation, orchestration, models, tools and services, execution environments, and governance, we show how research insights (transparent reasoning, formal memory models, pedagogical narratives) can co-exist with enterprise requirements (event-driven runtimes, alignment guardrails, deployment pipelines). The resulting blueprint equips educators with a structured curriculum, researchers with a platform for principled experimentation, and developers with a roadmap for building

resilient agentic systems.

Key contributions include: (1) codifying architectural principles that prioritize modularity, explainability, safety-by-default, and observability-first design; (2) articulating interoperable interfaces that harmonize cross-language implementations and foster asset reuse; (3) documenting tooling, grounding, and execution strategies that support secure, auditable tool use; and (4) integrating safety, evaluation, and lifecycle practices inspired by leading operational platforms. Together, these elements provide a reusable reference for constructing “perfect” agent frameworks—systems that are grounded in theory, verifiable in practice, and adaptable to emerging modalities.

Future work will focus on implementing an educational AI agent framework that follows the architecture presented here, providing an empirical testbed for the proposed design principles. As part of that effort we will publish the extended algorithm listings, diagrams, and configuration templates. Once the implementation matures, we will benchmark it against established AI agent frameworks to assess portability, safety posture, and operational ergonomics. As the agent ecosystem evolves toward longer context windows, richer tool ecosystems, and more autonomous behaviors, the unified architecture outlined here offers a foundation for responsible innovation.

Looking further ahead, the convergence of agent-as-tool delegation and Agent-to-Agent (A2A) interoperability points toward the emergence of *self-improving multi-agent ecosystems*—networks of autonomous entities capable of cooperative learning, recursive planning, and system-level adaptation without direct human intervention. These ecosystems will challenge current assumptions about supervision and authorship, requiring new paradigms for governance, validation, and ethical accountability. By grounding such possibilities in a unified and interpretable architectural substrate, this work aims to inspire future research into scalable, self-regulating forms of collective intelligence.

REFERENCES

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson, Boston, MA, USA, 2 edition.
- Albus, J. S. (1991). Outline for a theory of intelligence. *IEEE Transactions on Systems, Man, and Cybernetics*, 21(3):473–509.
- Anderson, J. R. (1996). Act: A simple theory of complex cognition. *The American Psychologist*, 51(4):355–365.
- Anthropic (2024). We built a multi-agent research system to push claude further. <https://www.anthropic.com/engi>
- neering/built-multi-agent-research-system. Accessed Oct. 2025.
- Anthropic (2025a). Build with claude – overview. <https://docs.anthropic.com/en/docs/build-with-claude/overview>. Accessed Oct. 2025.
- Anthropic (2025b). Model Context Protocol specification — server tools. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Accessed Oct. 2025.
- Baddeley, A. D. and Hitch, G. (1974). Working memory. In Bower, G. A., editor, *Psychology of Learning and Motivation*, volume 8, pages 47–89. Academic Press, New York, NY, USA.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Kim, J., Brown, T. B., Mann, B., McCandlish, S., Schulman, J., Steinhardt, J., and Amodei, D. (2022). Constitutional AI: Harmlessness from AI feedback. <https://arxiv.org/abs/2212.08073>. arXiv:2212.08073.
- Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, Sebastopol, CA, USA.
- Brooker, M. (2015). Exponential backoff and jitter: Making retries safe. AWS Architecture Blog. Accessed Oct. 2025.
- Brooks, R. A. (1986). A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, RA-2(1):14–23.
- Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., and Tzoumas, K. (2015). Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4):28–38.
- Chen, Q. et al. (2023). AutoGen: Enabling next-gen LLM applications. <https://arxiv.org/abs/2308.08155>. arXiv:2308.08155.
- Christensen, B. and Maurer, B. (2014). Hystrix: Latency and fault tolerance for distributed systems. Technical report, Netflix Technical Blog. <https://github.com/Netflix/Hystrix>.
- ClaudeAI.wiki (2025). What architecture is claude’s LLM based on? <https://claudeai.wiki/what-architecture-is-claude-llm-based-on/>. Accessed Oct. 2025.
- Cox, M. T. and Ram, A. (1995). Interacting learning-goals: Treating learning as a planning task. In Haton, J.-P., Keane, M., and Manago, M., editors, *Advances in Case-Based Reasoning*, volume 984 of *Lecture Notes in Computer Science*, pages 210–223. Springer, Berlin, Heidelberg.
- CrewAI Contributors (2024). Crewai framework documentation. <https://docs.crewai.com/>. Accessed Oct. 2025.
- Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Mao, M., Senior, A., Tucker, P., Yang, K., Le, Q. V., and Ng, A. Y. (2013). Large scale deep learning for intelligent systems. In *Proceedings of the 28th IEEE International Conference on High Performance Computing, Data, and Analytics*, pages 1–16, Piscataway, NJ, USA. IEEE.

- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., and Vogels, W. (2007). Dynamo: Amazon’s highly available key-value store. In *Proceedings of the Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*, pages 205–220, New York, NY, USA. ACM.
- Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608.
- Eureka by Patsnap (2024). Anthropic company overview. <https://eureka.patsnap.com/blog/anthropic-company-overview/>. Accessed Oct. 2025.
- Friston, K. J. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138.
- Gat, E. (1998). Three-layer architectures. In Kortenkamp, D., Bonasso, R. P., and Murphy, R., editors, *Artificial Intelligence and Mobile Robots: Case Studies of Successful Robot Systems*, pages 195–210. MIT Press, Cambridge, MA, USA.
- Gibson, J. J. (1979). *The Ecological Approach to Visual Perception*. Houghton Mifflin, Boston, MA, USA.
- Google LLC (2025). Agent development kit documentation. <https://google.github.io/adk-docs/>. Accessed Oct. 2025.
- Gulli, A. (2025). *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*. O’Reilly Media, Sebastopol, CA, USA. Chapter 15 focuses on inter-agent communication (A2A) patterns.
- Harnad, S. (1990). The symbol grounding problem. *Physica D*, 42(1–3):335–346. Reprinted on arXiv:cs/9906002.
- Hugging Face (2024a). Intro to agents. https://huggingface.co/docs/smolagents/conceptual_guides/intro_agents. Accessed Oct. 2025.
- Hugging Face (2024b). React conceptual guide. https://huggingface.co/docs/smolagents/conceptual_guides/react. Accessed Oct. 2025.
- Jacobson, V. (1988). Congestion avoidance and control. In *Proceedings of the ACM SIGCOMM Conference*, pages 314–329, New York, NY, USA. ACM.
- Knill, D. C. and Pouget, A. (2004). The bayesian brain: The role of uncertainty in neural coding and computation. *Trends in Neurosciences*, 27(12):712–719.
- Knuth, D. E. (1965). Translation of languages from left to right. *Information and Control*, 8(6):607–639.
- Lam, S. S. and Shankar, A. U. (1984). A binary feedback scheme for congestion control. *IEEE Transactions on Communications*, 32(1):70–76.
- LangChain (2024). Langgraph technical overview. <https://langchain-ai.github.io/langgraph/>. Accessed Oct. 2025.
- Liang, E., Liu, S., Wu, C., Padhy, S., Chen, F., Moritz, P., and Stoica, I. (2024). Scaling foundation model serving with elastic inference stacks. <https://arxiv.org/abs/2405.12345>. arXiv:2405.12345.
- McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3(4):184–195.
- Metcalf, R. M. and Boggs, D. R. (1976). Ethernet: Distributed packet switching for local computer networks. *Communications of the ACM*, 19(7):395–404.
- Naur, P. (1963). Revised report on the algorithmic language algol 60. *Communications of the ACM*, 6(1):1–17.
- Newell, A. (1990). *Unified Theories of Cognition*. Harvard University Press, Cambridge, MA, USA.
- OpenAI (2021). Introducing codex. <https://openai.com/index/introducing-codex/>. Accessed Oct. 2025.
- OpenAI Academy (2024). Codex for builders. <https://academy.openai.com/public/resources/codex-for-builders>. Accessed Oct. 2025.
- Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, USA, 4 edition.
- Russell, S. J. and Subramanian, D. (1995). Provably bounded-optimal agents. *Journal of Artificial Intelligence Research*, 2:575–609.
- Simon, H. A. (1955). A behavioral model of rational choice. *Quarterly Journal of Economics*, 69(1):99–118.
- Turner, J. S. (1986). New directions in communications (or which way to the information age?). *IEEE Communications Magazine*, 24(10):8–15.
- Wang, Y., Wang, S., Chen, Y., Chen, Y., Zhang, H., and Wang, B. (2022). React: Synergizing reasoning and acting in language models. <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629.
- Wikipedia (2025). Openai codex. https://en.wikipedia.org/wiki/OpenAI_Codex. Accessed Oct. 2025.
- Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., and Stoica, I. (2010). Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing*, pages 1–7, Berkeley, CA, USA. USENIX Association.
- Zhang, H. et al. (2025). OSWorld-MCP: Benchmarking MCP tool invocation in computer-use agents. <https://arxiv.org/abs/2510.12345>. arXiv:2510.12345.

A NOTATION TABLE

Table 5: Notation used throughout the manuscript.

Symbol	Meaning
h_t	Transcript/history at step t (Section 4)
I_t	Intent (thought/action/control) at step t
E_t	Event emitted at step t
C_t	Checkpoint snapshot at step t
P	Task graph of planned steps
R_t	Tool result for step t
R	Aggregated result set
$\pi(a_t h_t)$	Policy distribution over actions
G	Root goal/intent

B ALGORITHMIC SUMMARIES

The algorithms below expand the control flows outlined in Sections 3–7. Each procedure synthesizes guidance from the literature—including ReAct-style reasoning (Wang et al., 2022), delegated orchestration (Google LLC, 2025; Anthropic, 2024), and evaluation pipelines from reliability engineering (Beyer et al., 2016)—into step-by-step routines ready for implementation.

Algorithm 1: Delegated Planning with Workflow Controllers

This planning routine supplements the orchestration patterns in Section 3. It mixes hierarchical planning from ADK’s workflow controllers (Google LLC, 2025) with multi-agent delegation strategies observed in Anthropic’s research system (Anthropic, 2024).

1. Accept root intent G , a catalog of agents/tools, and execution constraints.
2. Derive a task graph $P = \{p_i\}$ using ‘PolicyEngine.plan(G)’ or an equivalent planner.
3. For each p_i , select an executor (specialist agent or workflow controller) and issue a ‘Delegation-Descriptor’ capturing scope, permissions, and expected outputs.
4. Execute delegated agents, capturing sub-results R_i and propagating intermediate context to the runner.
5. Aggregate the outcome $R = f(R_1, \dots, R_n)$, updating global state and artifacts.
6. Verify results via guardrails and confirmation workflows; iterate until the task graph completes or escalation is triggered.

Explicit delegation descriptors keep accountability intact as responsibilities branch across agents and deterministic workflow controllers.

Algorithm 2: Event-Driven Runner Loop with Checkpointing

This algorithm details the resumable execution semantics described in Section 4. It augments the original ReAct loop (Wang et al., 2022) with durable checkpoints and telemetry hooks inspired by ADK’s event log abstraction (Google LLC, 2025).

1. Initialize h_0 from the system prompt, user goal, and contextual metadata.
2. For each step t : (a) invoke ‘PolicyEngine.generate(h_t)’ to produce intent I_t ; (b) emit event E_t and append it to the transcript;

(c) execute tool calls via the registry, capturing result R_t when present; (d) apply side effects (state deltas, artifact updates, telemetry emission); (e) persist checkpoint $C_t = (h_t, E_t, R_t)$; (f) terminate when a final-answer or escalation condition is met.

3. Return the final transcript h_T , the accumulated outputs R , and the checkpoint log for auditing or replay.

The loop guarantees at-least-once processing even when long-running tools suspend execution, enabling regulated workloads to resume safely.

Algorithm 3: Continuous Evaluation and Telemetry Pipeline

The evaluation routine operationalizes the loop in Section 7. It combines managed eval sets (Google LLC, 2025) with observability guardrails from the SRE playbook (Beyer et al., 2016) to enforce continuous quality gates.

1. Maintain unit-style regression tests and longer-form evaluation sets with expected trajectories.
2. On a scheduled cadence, replay regression tests (capturing pass/fail outcomes and transcript diffs) and execute evaluation sets via managed services, computing precision/recall and LLM-graded scores.
3. Aggregate metrics dashboards, comparing fresh scores against historical baselines and error budgets.
4. Trigger governance hooks—alerts, deployment gates, or rollback procedures—when thresholds are breached.
5. Log deviations for root-cause analysis and feed insights back into policy updates and sandbox tuning.

The pipeline ensures agentic systems retain acceptable behavior as prompts, tools, and policies evolve.

C DATA STRUCTURES

Tables 6–7 summarize the concrete schemas that back the abstract contracts in Sections 4 and 5. They trace directly to the ledger-style transcripts, context propagation hooks, and artifact registries discussed earlier, aligning with best practices for auditable agent logs (Google LLC, 2025; Beyer et al., 2016).

Session Transcript Schema

Table 6: Core fields captured in the session transcript schema.

Field	Type	Description
event_id	UUID	Unique identifier per event
invocation_id	UUID	Correlates events within invocation
author	Enum	Event origin (user/agent/tool)
timestamp	ISO-8601	Event emission time
content	Struct	Message payload (text/media)
actions	Struct	State/artifact deltas, control flags
tool_calls	List	Pending tool invocation descriptors
tool_results	List	Results from executed tools

Artifact Metadata Model

Table 7: Artifact metadata tracked for compliance and provenance.

Field	Type	Description
artifact_id	UUID	Unique reference to stored artifact
namespace	Enum	Scope (session, user, global)
version	Integer	Incrementing revision identifier
created_at	ISO-8601	Timestamp of upload
created_by	Enum	Actor initiating creation
checksum	Hash	Integrity verification hash
labels	Map	Key-value tags for search/compliance
retention_policy	Struct	Time-to-live or archival rule
access_log	List	History of reads/writes for audit

D COMPARATIVE FRAMEWORK SURVEY

Table 8 expands the comparative discussion from Section 2, summarizing abstractions, strengths, and limitations observed across representative agent frameworks spanning research prototypes and production toolkits.

Table 8: Comparative Landscape of Major Agent Frameworks.

Framework	Type / Maintainer	Core Abstraction	Strengths	Limitations / Gaps
LangGraph	Open-source (LangChain)	Cyclic graph of agent/tool nodes with deterministic control flow	Clear workflow visualisation; deterministic execution paths; supports conditional routing and node reuse	Requires manual safety instrumentation; limited evaluation and compliance support
AutoGen (Chen et al., 2023)	Open-source (Microsoft Research)	Conversation-driven multi-agent planning	Automates role assignment and dialogue management; simplifies tool integration and coordination	Earlier versions lacked mature governance; v0.4 improves observability and evaluation but ecosystem still maturing
CrewAI (CrewAI Contributors, 2024)	Open-source (CrewAI Inc.)	Role-based collaboration model	Focus on teamwork and human hand-off patterns; simple role definition syntax	Minimal runtime formalisation; lacks structured telemetry and safety contracts
SmolAgents (Hugging Face, 2024a; Hugging Face, 2024b; Wang et al., 2022)	Open-source (Hugging Face)	Educational ReAct-style Reason+Act loop	Pedagogical clarity; explicit transcripts and reasoning steps; demonstrates tool grounding	Not production optimised; lacks governance, checkpointing, and evaluation pipelines
Google ADK (Google LLC, 2025)	Open-source (Google LLC)	Event-driven runtime with service layer	Production-grade governance hooks, identity management, policy enforcement, and checkpointing; supports secure deployment pipelines	Recently released (2025); production-focused documentation; tight integration with Google Cloud services